Labrorator 2 Algoritmi fundamentali
MEMORAREA GRAFURILOR. PARCURGERI. APLICAȚII

Implementați algoritmi eficienți pentru rezolvarea următoarelor probleme. Se vor adăuga comentarii în cod cu ideea algoritmului și complexitatea sa.

1. **Graf bipartit.** https://cses.fi/problemset/task/1668

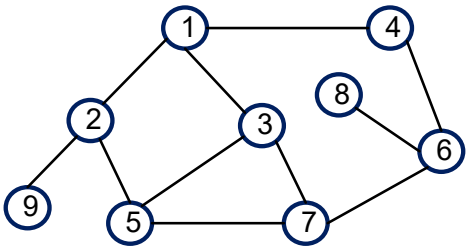
2. a) **Sortare topologică.** https://cses.fi/problemset/task/1679

b) Modificați programul de la a) astfel încât în cazul în care nu pot fi urmate toate cursurile (deci se afișează IMPOSSIBLE), să afișeze și o listă de cursuri [c_1, c_2, …, c_k, c_1] care depind circular unele de altele (orice curs c_i trebuie urmat înaintea cursului c_{i-1}, iar c_k înaintea cursului c_1).

3. **Fill** https://cses.fi/problemset/task/1192

4. **Componente tare conexe.** **https://cses.fi/Algoriproblemset/task/1683**

5. **Puncte critice (de articulație).** https://www.spoj.com/problems/SUBMERGE/

6. **Distanțe și drumuri minime**. Se dă o rețea neorientată cu n noduri și o listă de noduri reprezentând puncte de control pentru rețea în fișierul ***graf.in***, în formatul precizat la începutul laboratorului; în plus, pe ultima linie din fișier se află punctele de control separate prin spațiu. Să se determine pentru fiecare nod din rețea distanța până la cel mai apropiat punct de control de acesta. În fișierul ***graf.out*** se vor afișa pentru fiecare nod de la 1 la n aceste distanțe separate prin spațiu.

graf.in - graful alaturat; pe ultima linie: punctele de control	**graf.out**
9 11 1 2 1 3 1 4 2 5 2 9 3 5 3 7 5 7 6 7 6 8 4 6 **8 9**	2 1 3 2 2 1 2 0 0 **(Explicații:** **Punctele de control sunt 8 si 9** **- cel mai apropiat punct de control de nodul 1 este 9, aflat la distanță 2** **- cel mai apropiat punct de control de nodul 2 este 9, aflat la distanță 1** etc)